Arithmetic expressions

Type conversions

Chapter 7

Expressions and
Assignment Statements

Introduction

For imperative languages, variables, expressions and assignments are fundamental concepts. Variables are abstractions of memory cells. Computations are realized by evaluating expressions and assigning resulting values to variables, thus changing machine states.

Arithmetic Expressions

- The value of an arithmetic expression largely depends on
  - the operator precedence rules
  - the operator associativity rules
  - the order of operand evaluation

Operator Precedence Rules

- Define the order in which the operators of different precedence levels are evaluated

- Typical precedence levels

  parentheses

  ** (if the language supports it)

  unary operators

  *, /, %

  +, -

Operator Associativity Rule

- Define the order in which adjacent operators with the same precedence level are evaluated
- Typical associativity rules
  - Left to right, except **, which is right to left
    - Left to right: A + B + C $\longrightarrow$ (A + B) + C
      - 72 / 2 / 3 is treated as (72 / 2) / 3
    - Right to left: A + B + C $\longrightarrow$ A + (B + C)
      - x = y = z = 17 is treated as x = (y = (z = 17))
    - Fortran exponentiation: A ** b ** C $\longrightarrow$ A** (B ** C)
    - Java: For example x = y = z = 17 is treated as x = (y = (z = 17))

Operand Evaluation Order

- The order in which operands are evaluated may affect the resulting value.

```
foo(int& x) {
        x++;
        return x;
}
int main(void) {
        int a = 4;
        int b = a + foo(a);
}
```

```
int a =5;
int fun1()
{
    a =17;
  return 3;
}
void main()
{
    a =a + fun1();
}
```

- The operands may be evaluated right-to-left →

  b = 10, a = 20

- The operands may be evaluated left-to-right →

  b = 9, a = 8

Functional Side Effects

- when a function changes a parameter or a non-local variable

a = a + fun1(a)
int b = a + foo(a);

  - If **fun1 or foo** does not have the side effect, then the order evaluation of the two operands should not matter
  - If fun does have the side effect, order of evaluation matters

```
foo(int& x) {
    x++;
    return x;
}
int main(void) {
    int a = 4;
    int b = a + foo(a);
}
```

```
int a =5;
int fun1()
{
    a =17;
 return 3;
}
void main()
{
    a = a + fun1();
}
```

Functional Side Effects/2

- Two possible solutions to the problem
  1. Write the language definition to disallow functional side effects
     - No two-way parameters in functions
     - No non-local references in functions
     - **Advantage:** it works!
     - **Disadvantage:** inflexibility of two-way parameters and non-local references
  2. Write the language definition to demand that operand evaluation order be fixed
     - **Disadvantage**: limits some compiler optimizations

Other functional side effects

A function

- might modify a global or static variable,
- modify one of its arguments,
- raise an exception,
- write data to a display or file,
- read data,
- or call other side-effecting functions.